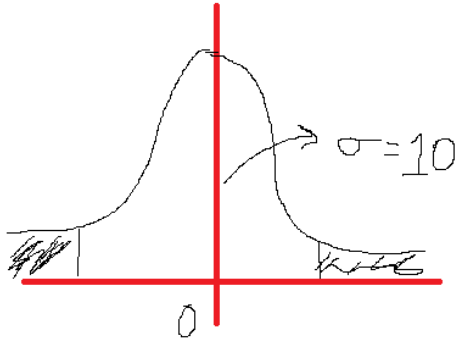


Averaging Filter on a Noisy dataset

Github (for code): https://github.com/pandaiscodingcpu/autonomous_navigation

Assumptions made in the synthetic dataset:

1. The dataset consists of 2000 readings at a rate of 1Hz.
2. External noise is added with mean: 0 and Standard deviation (S.D/sigma): 10



**The noise follows a Normal Distribution ($\mu = 0$ and $\sigma = 10$)

3. Since, the data here is synthetic the perfect effect of oversampling cannot be demonstrated without adding complexity. Hence, oversampling is not demonstrated.
4. Idea is to just demonstrate the difference between Raw noisy readings and averaged out readings. (Code is not included due to the limitations of the scope of this document, only results are shown)

An Important thing to understand:

Handwritten notes explaining the dataset and the averaging process:

Dataset = $\begin{bmatrix} \text{Pkt} & \text{Alt} \\ 1 & x_1 \\ 2 & x_2 \\ 3 & x_3 \\ \vdots & \vdots \\ 2000 & x_{2000} \end{bmatrix}$

Working with only [Alt]

Adding the Gaussian Noise

$\text{Alt} = \text{Alt} + \text{np.random.normal}(0, 10, n)$

$n = \text{len}(\text{Alt})$

Why $\mu = 0$? $\rightarrow$ So that noise is centered around 0.

Why s.d = 10? $\rightarrow$ To avoid "ideal" noise which is far from reality.

@10Hz (Pop!)

We take $[x_1, x_2, \dots, x_{10}] \Rightarrow \bar{x}_1$

$[x_{12}, x_{13}, \dots, x_{21}] \Rightarrow \bar{x}_2$

$\vdots$

$[x_{1990}, \dots, x_{2000}] \Rightarrow \bar{x}_{200}$

Like this we will get 200 sampled values. ($\frac{2000}{10} = 200$)

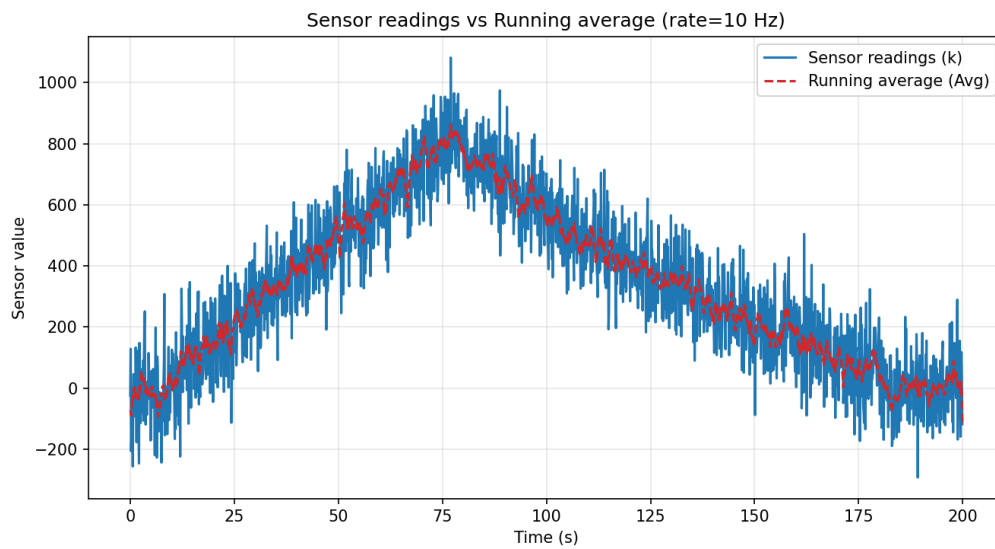
Similarly @ 100Hz $\rightarrow$ 20 sample values.

Hence in the results, one might notice that @10Hz avg. filtered values are more real than @100Hz.

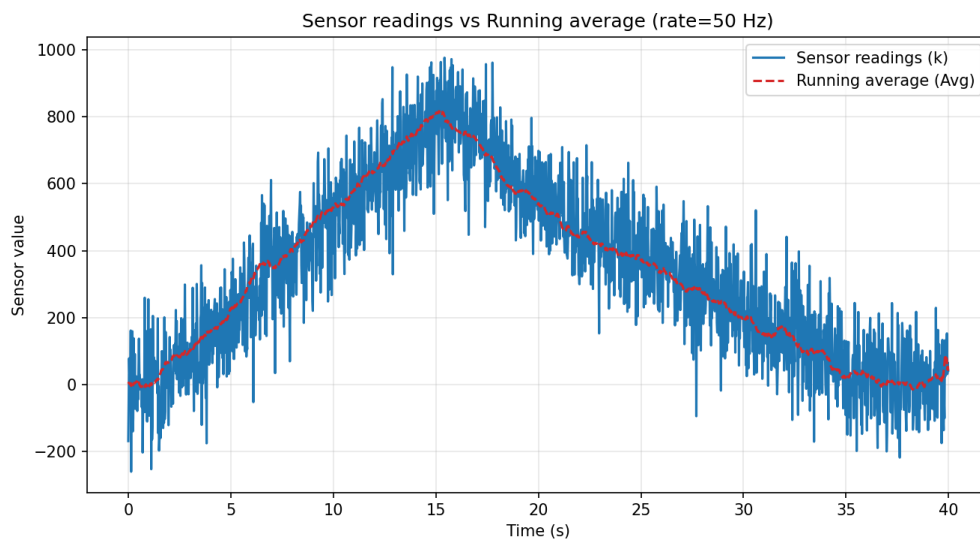
But, in real-life @100Hz will give more accurate values. It's just that 2000 pkts. is fixed hence we don't have enough data @100Hz to sample.

If we had real incoming values it would have worked as expected.

A. @10 Hz (from A:C, $\sigma = 100$)



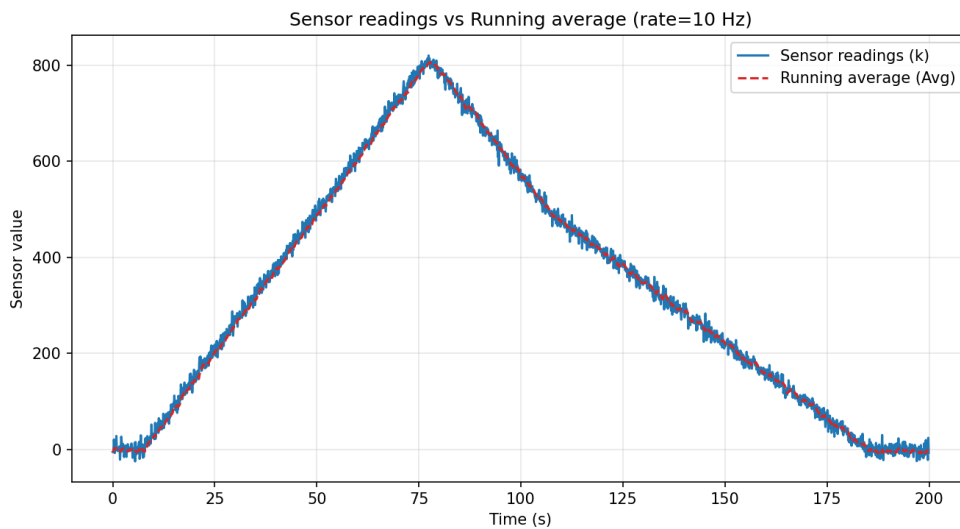
B. @50 Hz



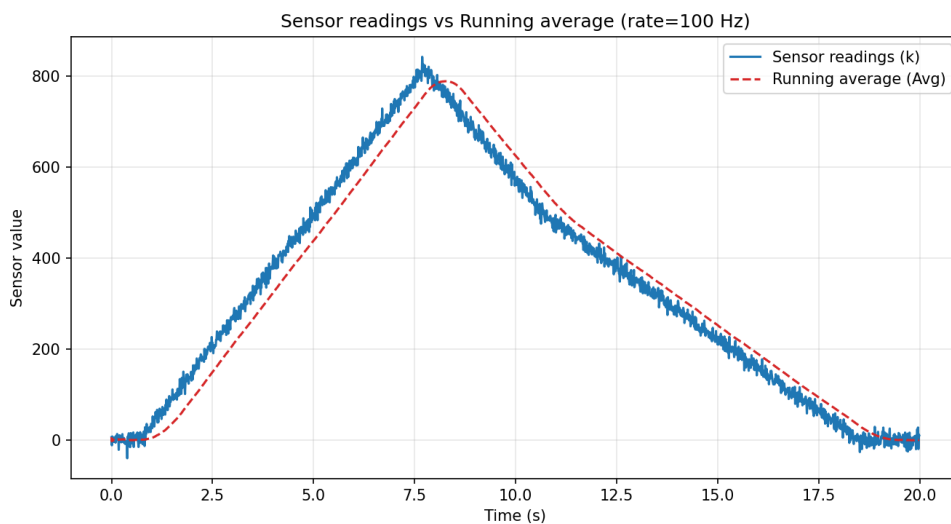
C. @100 Hz



D. @10 Hz



E. @100 Hz



What does this all mean?

Plot A: At 10 Hz, red line is well fit within the actual sensor readings. Apogee area using filter is same as apogee area as raw readings (peak of the triangle). Note the X-axis's readings (200)

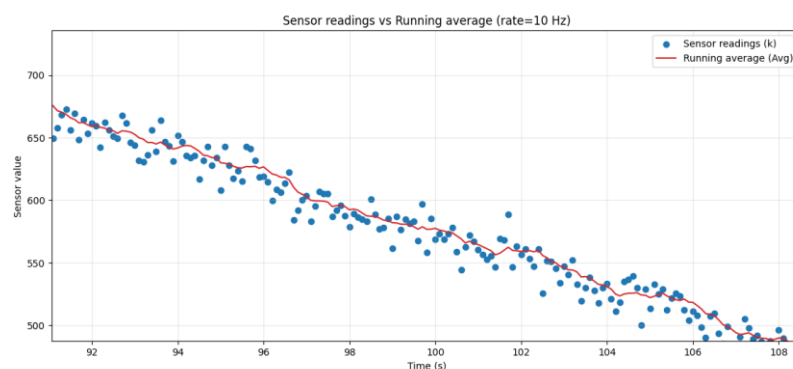
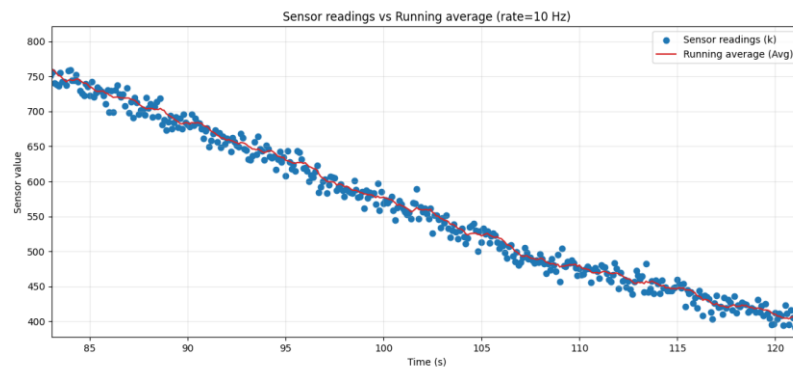
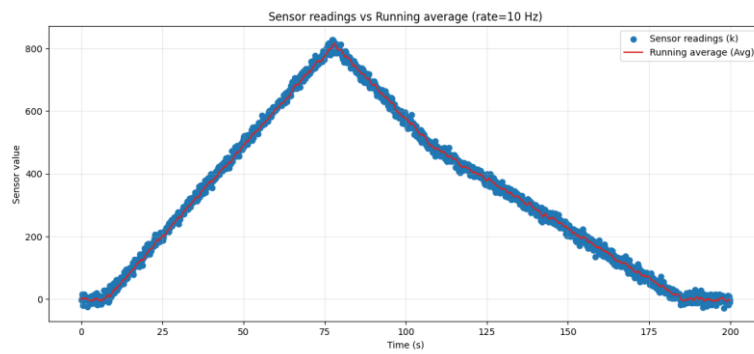
Plot B: At 50 Hz, though there is a deviation of peak from filtered value and actual reading, but still acceptable. Note the X-axis's readings (40). This is what the handwritten note is implying.

Plot C: At 100 Hz, the deviation is significant showing, the absence of enough data to sample at that rate. Note the X-axis's readings (20). Hence as we increase the rate, keeping the csv length fixed, we will keep getting less and less sampled values and eventually that will be not accurate or even near to the real sensor readings.

Plot D: As the SD is reduced to 10 from 100, plot is much cleaner, and same as Plot A, it is very well in accordance to the sensor readings.

Plot E: Even after reducing the SD to 10, @100 Hz, deviation is seen, reason is same as Plot C.

Zoomed-in view of the plot (@10 Hz, SD=10)



Conclusion: The averaging filter works as expected, the only drawback is that the fixed CSV of 2000 pkt. is restricting to demonstrate the oversampling of values. **Oversampling is a method of smoothening (removing noise/spikes) from the sensor readings to get much more accurate values over a period of time. It involves sampling sensors at a much higher rate than required. Less the oversampling rate, poor is the smoothenin**

LOW PASS FILTER

***Assumptions are same as in the previous test**

What is a Low Pass Filter: Filters out High Frequency noise, only allowing low frequency data for calculations

Mathematically:

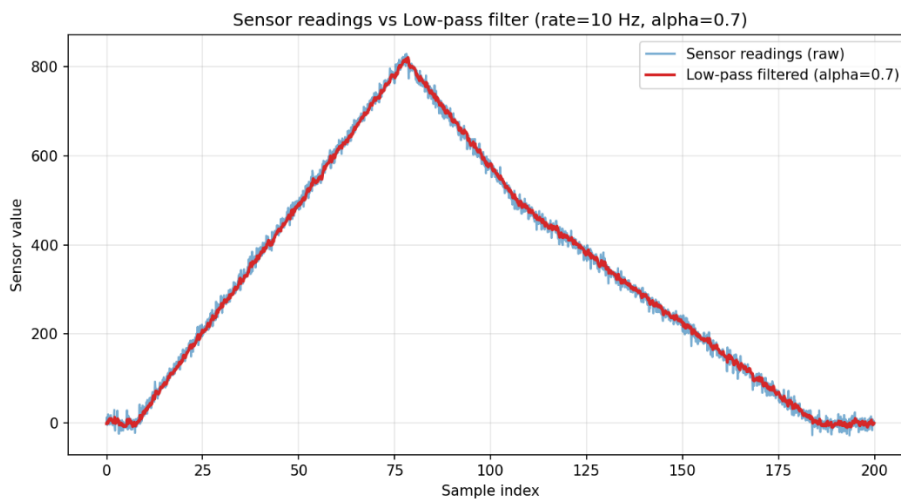
$$\overline{x_k} = \alpha \cdot \overline{x_{k-1}} + (1 - \alpha) \cdot x_k$$

Idea is to take a free variable (weight), and multiply it by previous (k-1) mean and add it to current value by multiplying it to (1 - α).

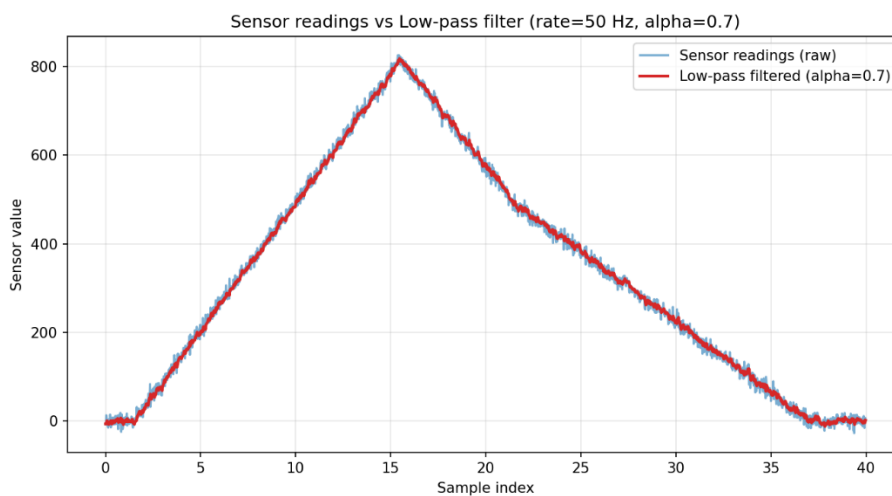
Basically, we assign more weight to the latest reading, instead of assigning equal weight to every reading in case of rolling avg. filter (self-explanatory).

Results:

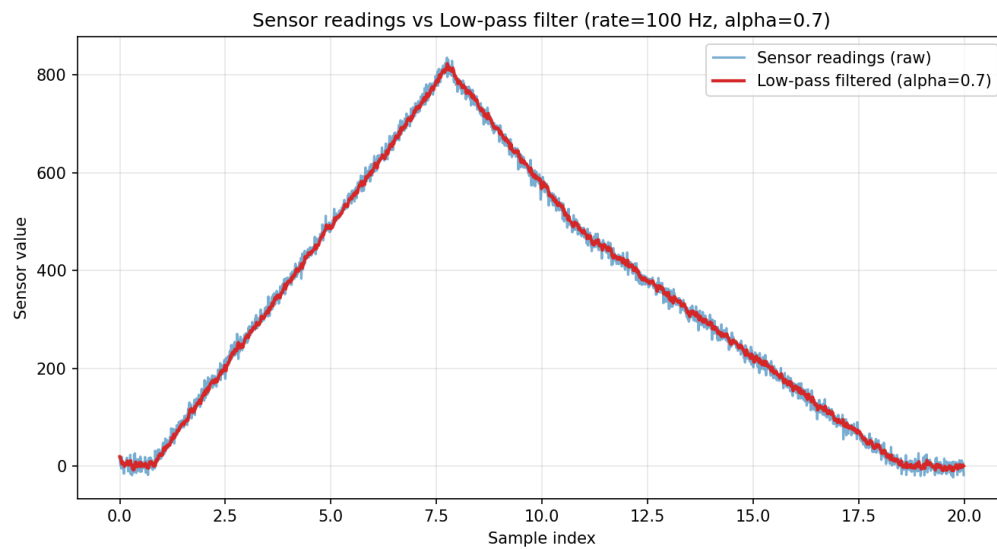
A. $\alpha = 0.7$ @ 10 Hz.



B. $\alpha = 0.7$ @ 50 Hz.



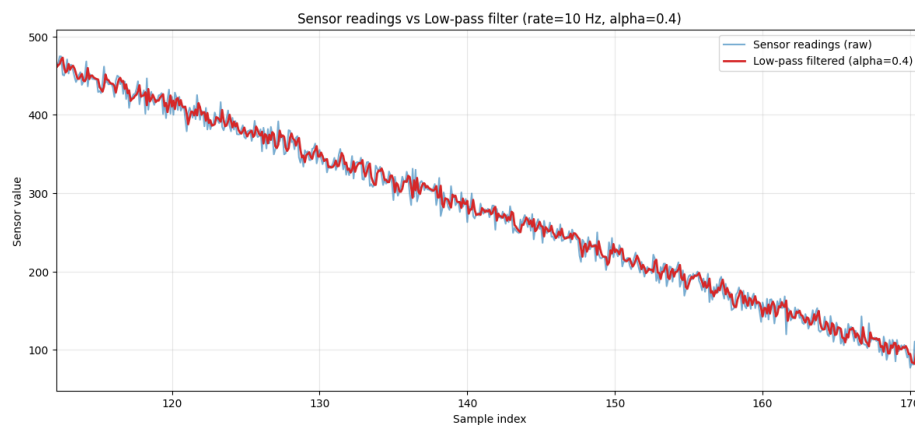
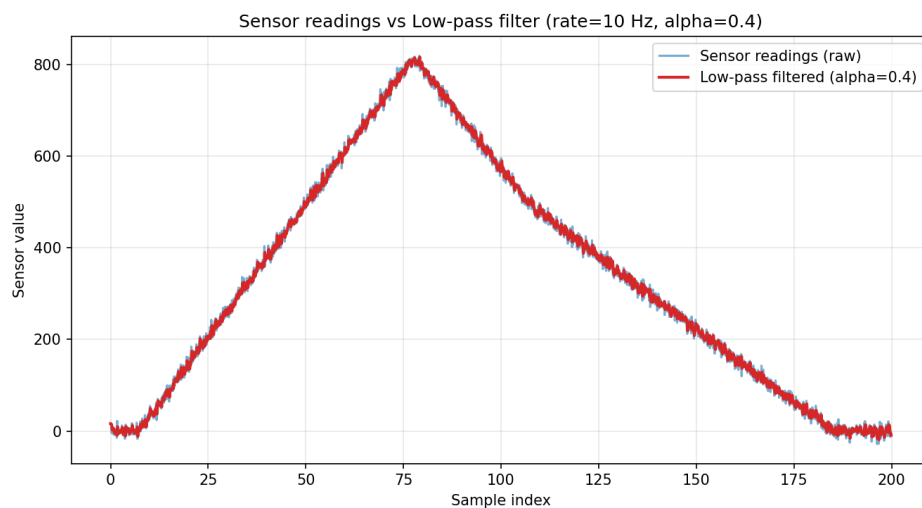
C. $\alpha = 0.7 @ 100 \text{ Hz}$



Now, let's see how alpha changes the predictions

TL;DR More Alpha, More accuracy and vice-versa

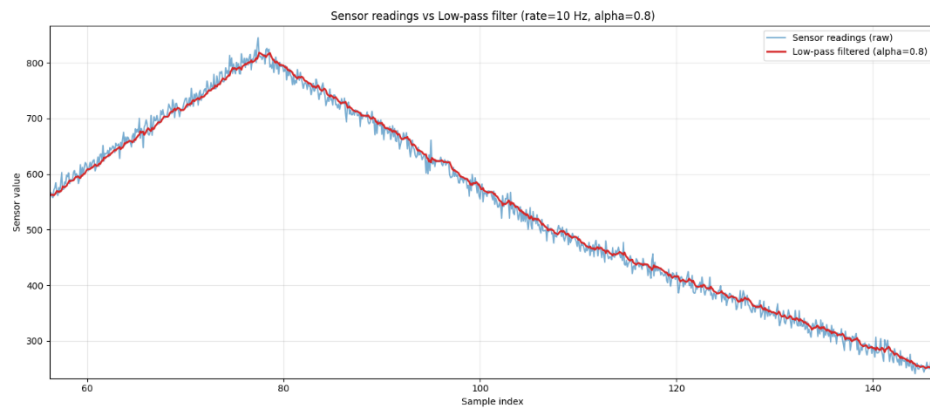
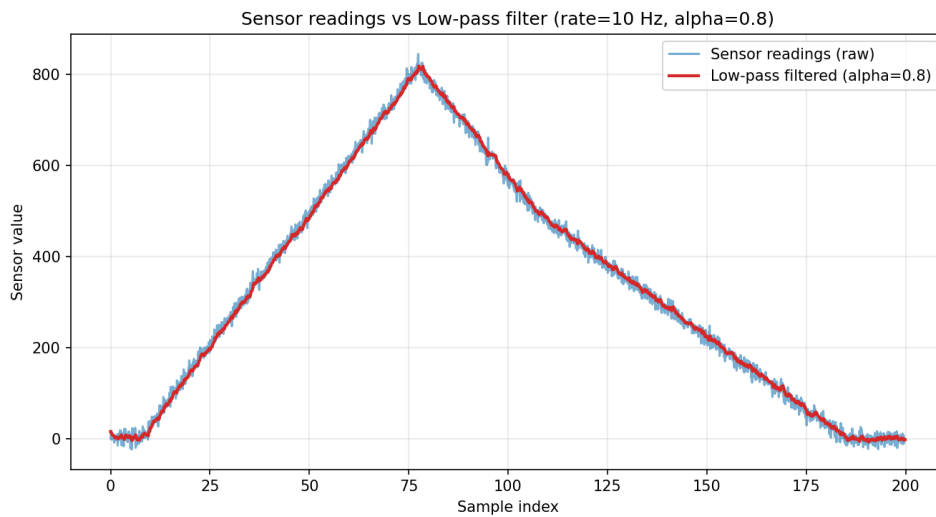
A. $\alpha = 0.4 @ 10 \text{ Hz}$





As in the plots, after decreasing the α , the smoothening effect is not as good as with 0.7.

B. $\alpha = 0.8 @ 10 \text{ Hz}$





Conclusion, Low pass filter performs comparatively better than rolling average, but the end result remains same. It doesn't matter what we choose, instead our use case is what that matters.

Next Steps: Implementation of Kalman Filter to predict velocity, from altitudes and using system states.

Kalman Filter

TL;DR

Kalman filter is basically a LPF with dynamic alpha. Meaning?

Step 01: We initialize our system states (in this case say, position) and a covariance matrix (a diagonal matrix, with variances among the system states at its diagonal, why? [I DON'T KNOW])

Step 02: We predict the model i.e we start with some prediction, How do we predict?

Well, there are some model variables (*model variables are either noise that we assume, the error rate etc*) usually these are represented as: A, R, Q, H (all these variables are vectors/matrices)

A: A State to transition matrix, one can say, it is some sort of "rule matrix" that governs how a particular state is transitioned over a period of time.

R: It is the noise level in the system, easy language? – Tells the model, how much to trust the sensor readings. More R, Less trust on sensor

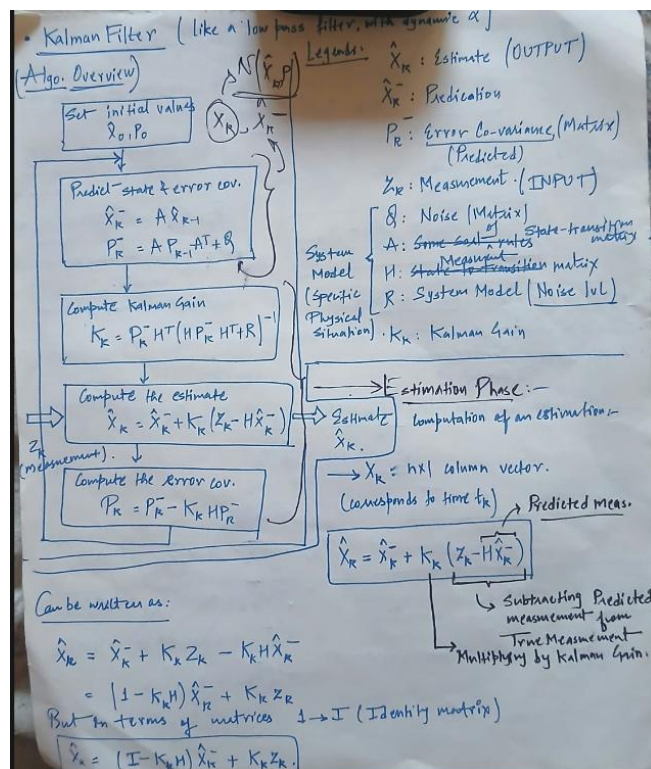
Q: This is the process noise, it depicts the noise in the readings.

H: This is the measurement noise, it tells how much noise to consider in the measurement of a reading.

Step 03: We then compute something called Kalman Gain, this is the main thing that makes the model improve over the period of time and improve its estimation.

Step 04: And at last, we compute the covariance matrix again, since we started with an assumption and in the first run, our covariance matrix (P) tells us how confident we are with our current readings, so with new readings, we of course need to shrink/reduce the matrix because now we have actual readings for 2nd run.

This continues.....



But the question still remains, what does Kalman filter do?

In short, it filters out the noise from the readings/signals and also helps us to estimate some unknown variable (say velocity) from a known variable (Eg. Position) without even needing to calculate it.

Yes, traditional method of taking derivative of position w.r.t time remains the same, it is just that Kalman filter is more optimized way to doing the same thing, with a view to reduce the delay in the readings and removing noise at the same time.

How do we calculate velocity, provided we are given only altitudes and a sensor rate?

Yes, again the answer is same, we take time derivative of position, but will that work? No, because the position's readings will itself have noise. There comes in Kalman filter, smooths out the readings estimates the velocity, and outputs position, velocity together.

• Estimating velocity ~~same~~ position (m=1) [No. of params]

→ Setting up the system model:

$$X_k = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}_k, \quad X_{k+1} = A \cdot X_k + w_k$$

time stamp blue mess. $Z_k = H \cdot X_k + V_k$

$$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}_{(2 \times 2)}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix}_{(1 \times 2)}$$

→ $X_{k+1} = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}_k$

→ $X_{k+1} = \begin{bmatrix} \text{position} + \Delta t \cdot \text{velocity}_k \\ \text{velocity}_k \end{bmatrix}$

→ $Z_k = H X_k + V_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix}_k + V_k$

→ $Z_k = [\text{position}]_k + V_k$

Noise: $Q = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$ (Assume made-up), $R = \begin{bmatrix} R=10 \end{bmatrix}$

How A & H are cal?

• Newton's Eq 1:

$$S_{k+1} = S_k + \Delta t \cdot V_k$$

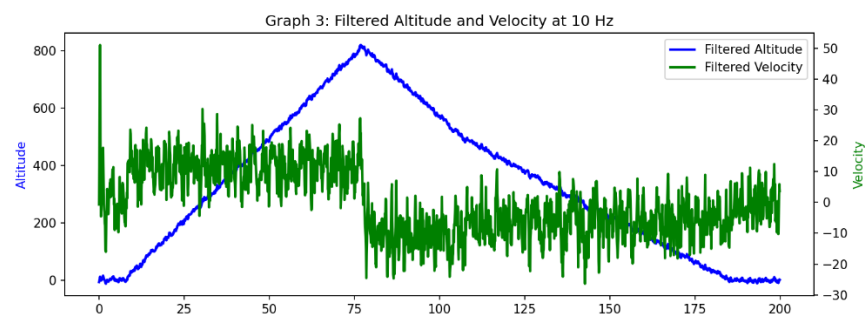
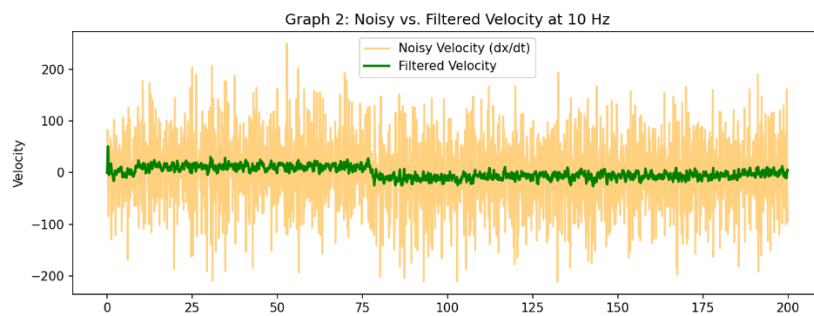
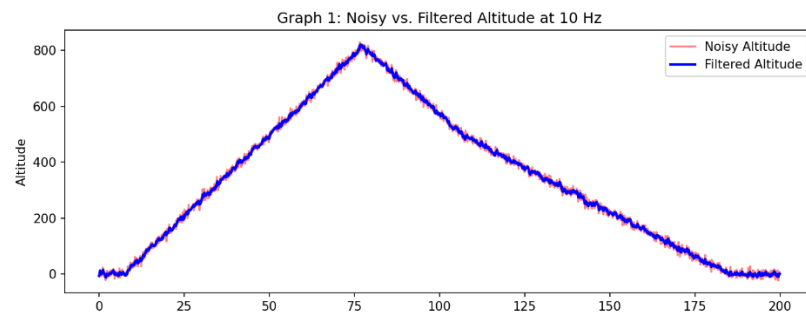
$$V_{k+1} = V_k$$

$$V_{k+1} = \frac{S_{k+1} - S_k}{\Delta t} = \frac{S_k + \Delta t \cdot V_k - S_k}{\Delta t} = \frac{\Delta t \cdot V_k}{\Delta t} = V_k$$

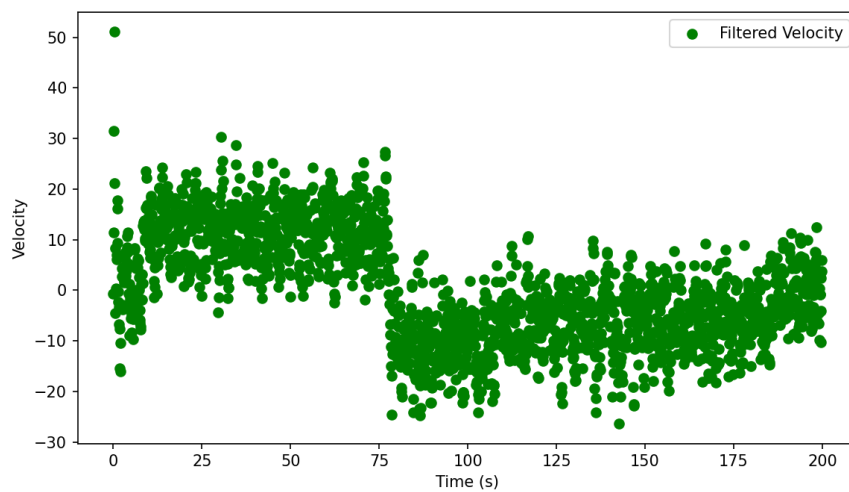
$$\begin{bmatrix} S_{k+1} \\ V_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} S_k \\ V_k \end{bmatrix} = A \cdot \begin{bmatrix} S_k \\ V_k \end{bmatrix}$$

**For results, the assumptions remain the same*

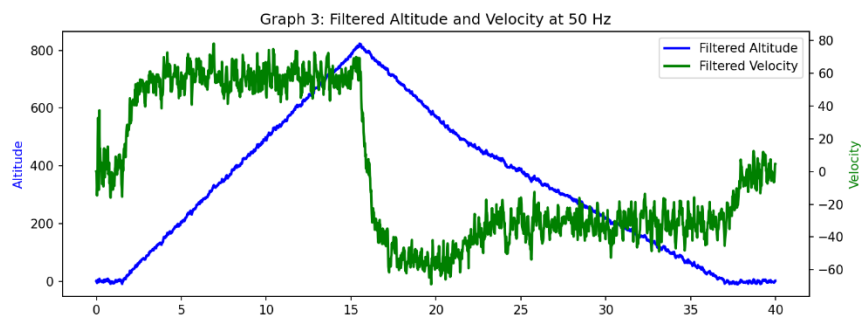
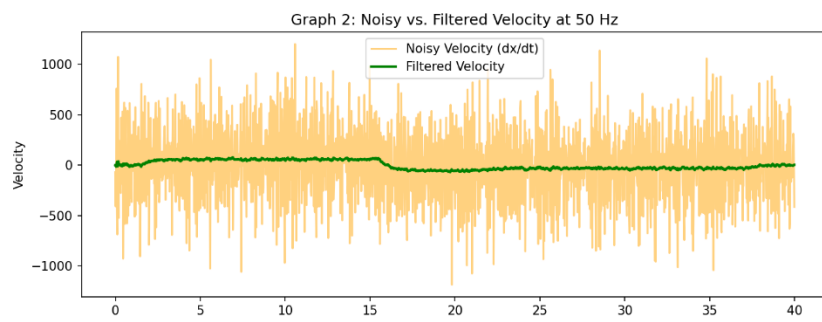
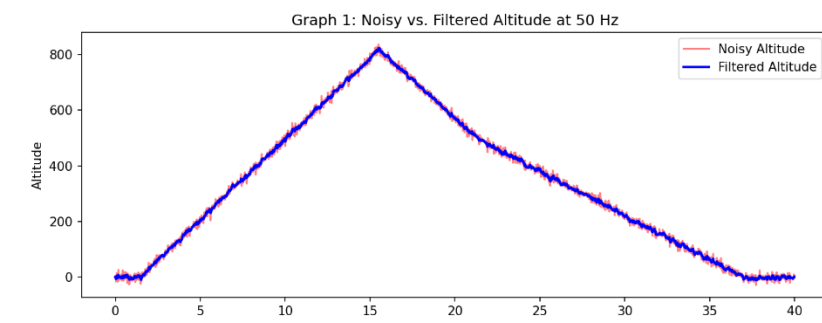
A. $\sigma = 10$ @ 10 Hz



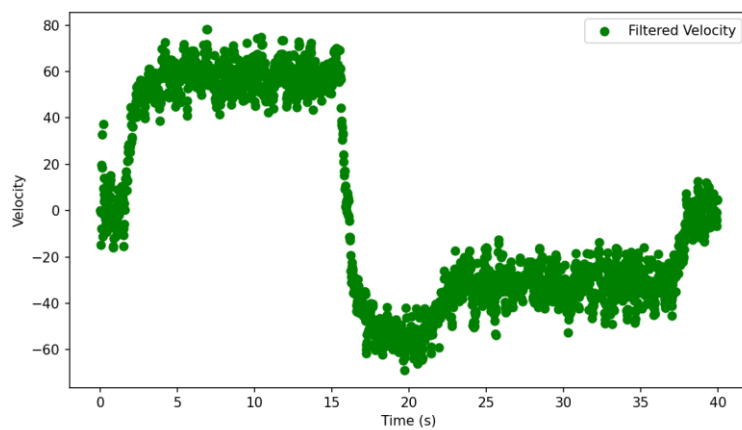
Velocity:



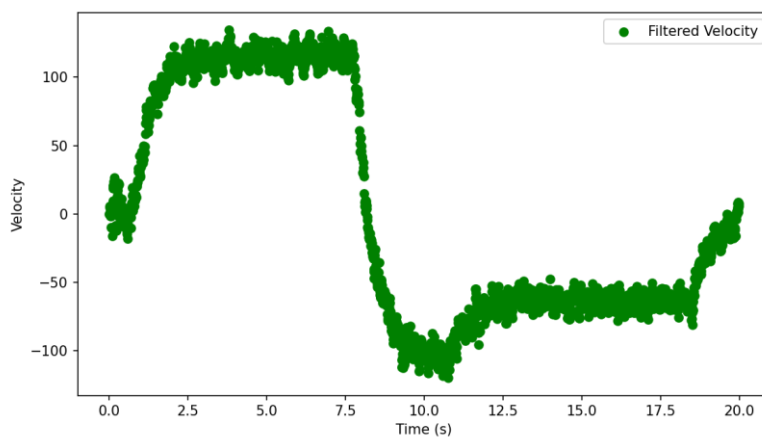
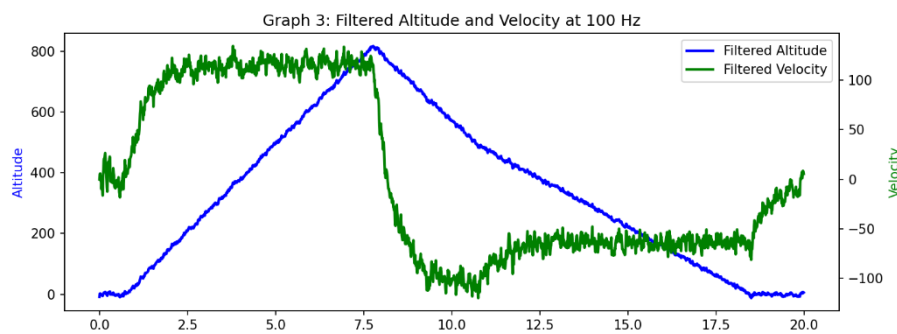
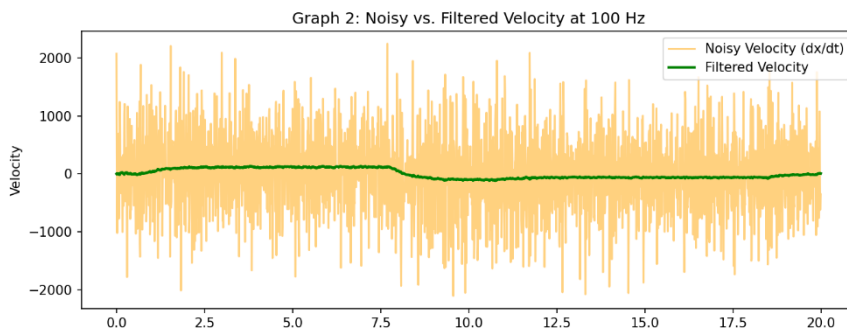
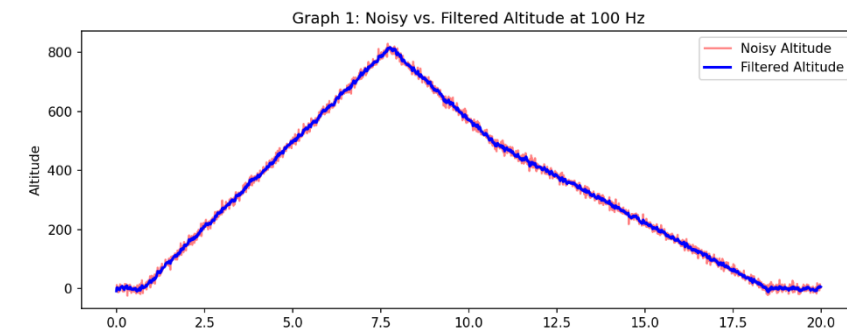
@50 Hz:



Velocity

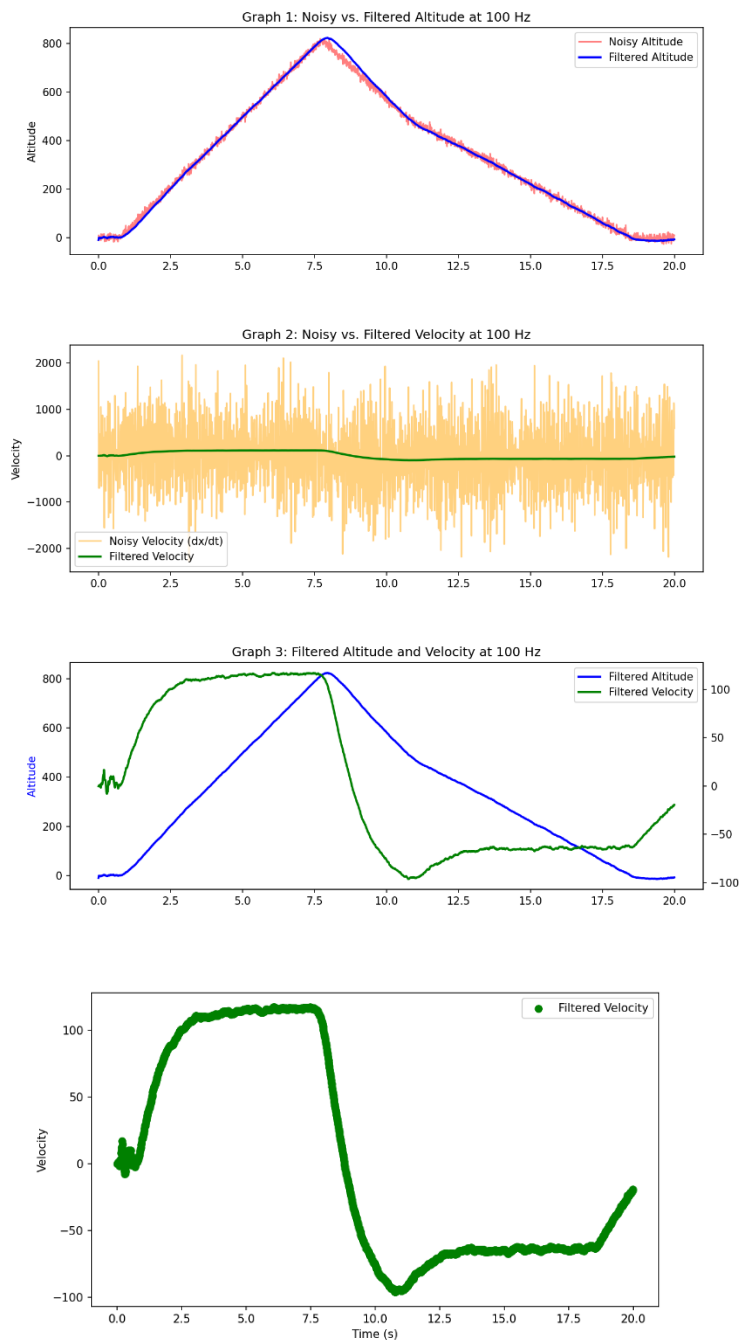


@ 100 Hz



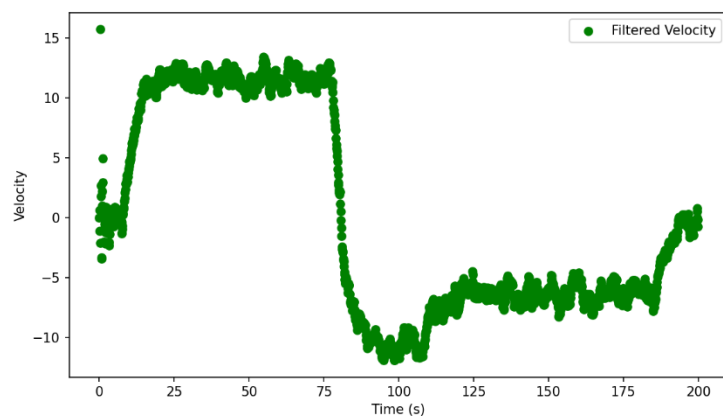
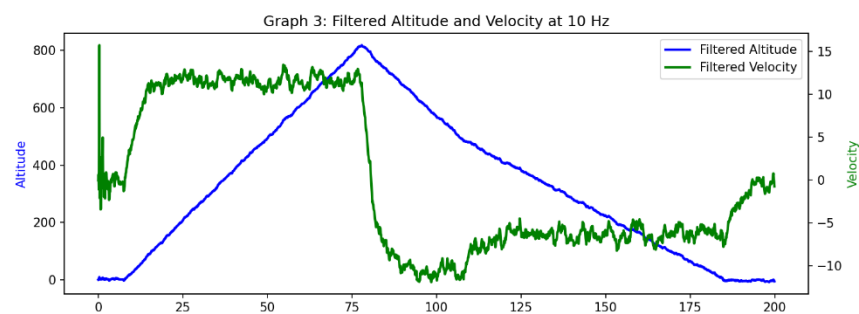
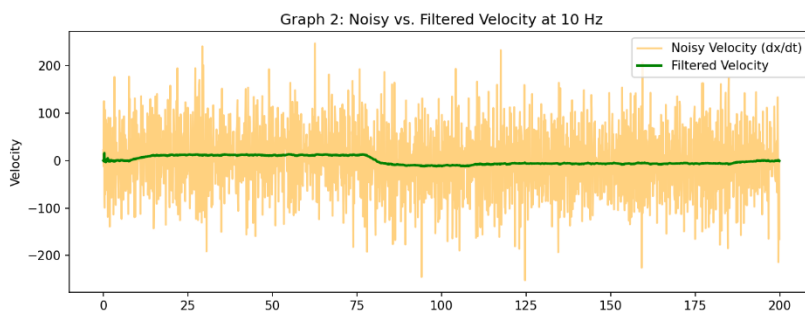
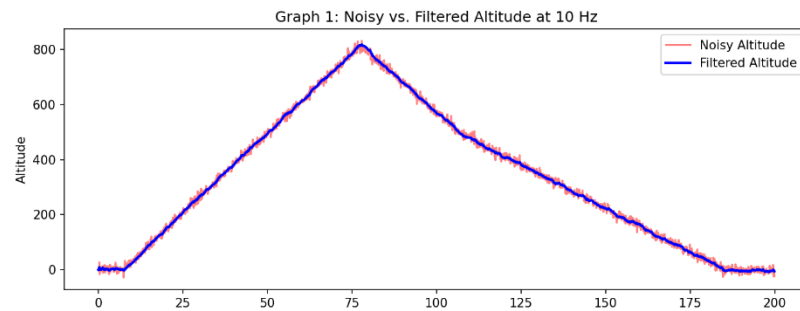
Conclusion: As we can see for all the middle graph, velocity has spikes at low frequencies, but as we increase the sampling rate it smooths better. Now, it can even get smoother if we tune the Model variables. In next plots, Q and R have been tuned. A and H are same.

Smoothed out @ 100Hz



The performs is improved, the estimation of velocity is far smoother than previous observation at 100 Hz, by tuning the Q and R matrices, this effect is observed even at low sampling rate ie @ 10Hz.

Smoothed Out @ 10Hz.



Now, here if we carefully, see and compare the prev. 10 Hz plots and this one, obv. This one is better but if we compare this 10 Hz with our tuned 100 Hz.

100 Hz sampling gives best results among all the observations.

So, we have seen how Kalman filter is used to remove noise and estimate a system variable without actually measuring it. (We can even estimate, Altitudes and Velocity using Accelerometer and Gyrometer data)

Applications are vast: Signal Processing, Navigation, Image processing etc.

@100 Hz, $\sigma = 10$:

$$A = \begin{pmatrix} 1 & dT \\ 0 & 1 \end{pmatrix}, Q = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.1 \end{pmatrix}, H = (1 \quad 0), R = (100), P = 500 * \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$